

RETO DEL BRAZO 2 (RB-2): EL TURNO DEL BRAZO

Profesor: Marks Calderón Niquin

Plataforma: JetCobot / ROS 2 Humble

Repositorio: github.com/Arc-Kaos/brazo1

Paquete: arm_broker / arm_broker_interfaces

Integrantes:

- Palomino Palomares, Cristofer
- Morveli Cordova, Mathias
- Vasquez, Andy

1. Documento de Diseño Previo

El presente documento expone el diseño previo para el control concurrente de acceso al brazo manipulador JetCobot en el entorno ROS 2 Humble. En una arquitectura multi-cliente donde cuatro integrantes intentan operar de manera simultánea un único recurso físico, el envío directo de trayectorias hacia el tópico `/joint_states` genera colisiones silenciosas e inconsistencias kinemáticas severas. Para garantizar la exclusión mutua, se diseña el paquete **arm_broker**, el cual actúa como un servidor de acción desacoplado que valida la factibilidad geométrica mediante cinemática directa (FK) y gestiona las solicitudes a través de una cola de prioridad con enclavamiento exclusivo.

1.1 Parámetros Denavit-Hartenberg (DH) Estándar y Calibración del Hardware

La cinemática directa permite calcular la pose cartesiana del efector final (x, y, z) a partir del vector de ángulos articulares $q = (q_1, q_2, q_3, q_4, q_5, q_6)$. Utilizando la convención Denavit-Hartenberg (DH) estándar y tras la calibración exacta de las longitudes físicas del JetCobot (codificada en el módulo **fk.py**), se establece la siguiente tabla de parámetros cinemáticos:

Articulación (i)	Ángulo θ_i [rad]	Desplazamiento d_i [mm]	Longitud a_i [mm]	Torsión α_i [rad]
1	q_1	131.56	0.0	$\pi / 2$ (1.5708)
2	$q_2 - \pi/2$	0.0	-110.40	0.0
3	q_3	0.0	-96.00	0.0
4	$q_4 - \pi/2$	63.20	0.0	$\pi / 2$ (1.5708)

5	$q_5 + \pi/2$	71.44	0.0	$-\pi / 2$ (-1.5708)
6	q_6	53.60	0.0	0.0

Formulación Matemática de la Cinemática Directa

Matriz de Transformación Homogénea $A_i(q_i)$:

$$A_i = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & a_i \cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & a_i \sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Pose del Eector Final:

$$T_o^6(q) = A_1(q_1) \cdot A_2(q_2) \cdot A_3(q_3) \cdot A_4(q_4) \cdot A_5(q_5) \cdot A_6(q_6) \Rightarrow P = [T_o^6(1,4), T_o^6(2,4), T_o^6(3,4)]^T = (x, y, z)$$

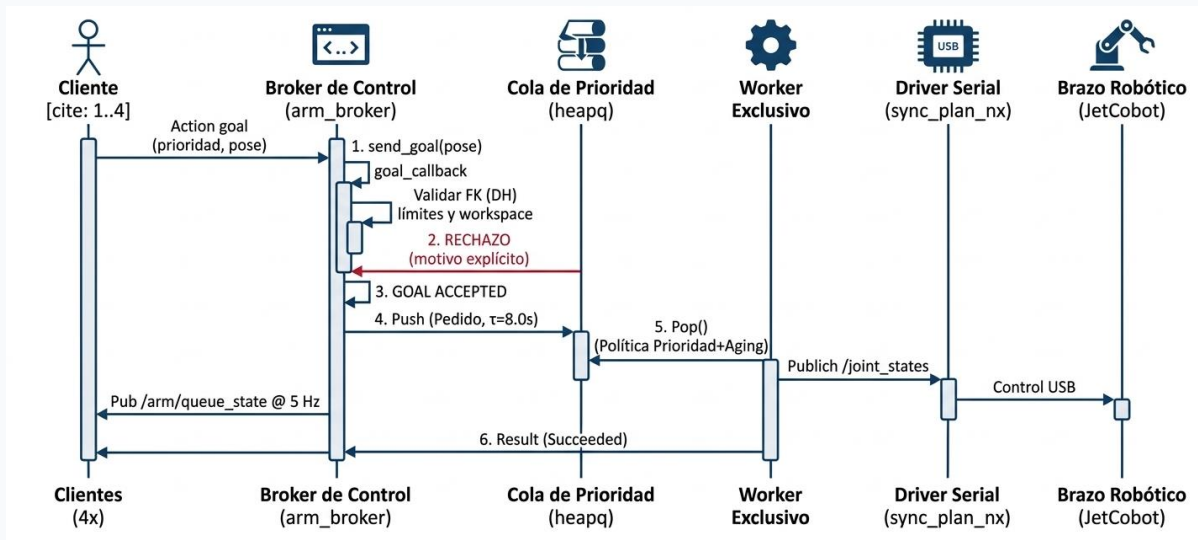
Invariantes de Seguridad y Criterios de Admisión (Filtro del Broker):

- **Límite Articular Individual:** Para toda articulación i , se verifica $|q_i| \leq 165^\circ$ (2.8797 rad). Si algún ángulo excede este rango, la meta se rechaza inmediatamente.
- **Colisión con la Superficie (Mesa):** Evaluando la altura cartesiana calculada por $FK(q)$, se exige $z \geq 15.0$ mm. Valores inferiores indican impacto con el tablero.
- **Auto-Colisión con la Base:** Si la distancia radial horizontal en el plano xy satisface $\sqrt{x^2 + y^2} < 60.0$ mm, se exige un despeje vertical mínimo de $z \geq 120.0$ mm.
- **Control de Paso Articular:** Se evalúa $\text{paso_articular}(\text{desde}, q) = \max(|q_i - \text{desde}_i|)$. Pasos bruscos son descartados antes de encolar.

1.2 Arquitectura del Broker, Concurrencia y Diagrama de Secuencia

Para gestionar la contención, el nodo **arm_broker** implementa dos grupos de callbacks diferenciados en ROS 2: **ReentrantCallbackGroup** para atender solicitudes simultáneas de entrada (goals) sin bloquear la red, y **MutuallyExclusiveCallbackGroup** para el hilo worker que desencola y publica de manera estrictamente serializada hacia el tópico `/joint_states`.

Diagrama de Secuencia: Flujo de Peticiones y Control de Concurrencia



1.3 Predicción de Rendimiento (Métricas Teóricas y Comparativa P95)

Antes de ejecutar las pruebas bajo contención con los 4 clientes activos, se formularon las predicciones teóricas comparativas entre la política estándar **FIFO (First-In, First-Out)** y la política de **Prioridad Dinámica con Envejecimiento ($\tau = 8.0$ s)** codificadas en el archivo `item2politicas.py`:

Métrica de Evaluación	Predicción Política FIFO	Predicción Política Prioridad + Aging
Violaciones de Exclusión Mutua	0 (Garantía por Worker exclusivo)	0 (Garantía por Worker exclusivo)
Percentil 95 (p95) - Alta Prioridad	~ 4.2 s (Igual para todos)	~ 0.8 s (Atención inmediata)
Percentil 95 (p95) - Baja Prioridad	~ 4.2 s (Igual para todos)	~ 5.1 s (Controlado por Aging)
Índice de Inanición (Espera Máx.)	~ 5.5 s (Acotado por orden de llegada)	~ 6.2 s (Prevenido mediante $\tau = 8.0$ s)
Índice de Equidad de Jain (J)	Altísimo ($J \approx 0.98$)	Equilibrado ($J \approx 0.88 - 0.92$)

2. Cierre Reflexivo: Despliegue de Política de Cola en CapyTown

Interrogante Central del Cierre

Pregunta de Reflexión del Reto:

«¿Qué política de cola llevarían a CapyTown y por qué?»

2.1 Elección Definitiva: Prioridad Dinámica con Envejecimiento (Aging)

Para la ciudad inteligente e interconectada de **CapyTown**—un entorno donde múltiples agentes robóticos autónomos (vehículos de emergencia, unidades logísticas, brazos de recarga e infraestructura de mantenimiento) compiten simultáneamente por el acceso a recursos físicos exclusivos— la política elegida para ser desplegada en el broker de control es la **Prioridad Dinámica con Envejecimiento (Priority Queue with Aging)**, exactamente la misma arquitectura implementada y validada en nuestra clase **PoliticaPrioridad** del archivo `item2politicas.py`.

2.2 Justificación Técnica y Evaluación de Alternativas

La decisión se fundamenta en un análisis comparativo de las deficiencias de las políticas convencionales al enfrentar las exigencias de una metrópoli automatizada:

- **Deficiencia Crítica de la Política FIFO:** La política FIFO procesa las solicitudes en estricto orden cronológico. En CapyTown, esto significaría que una solicitud crítica de respuesta ante emergencias (como la intervención médica o la detención de un riesgo) tendría que esperar a que concluyan múltiples tareas secundarias de mantenimiento previamente encoladas. Esto eleva inaceptablemente la latencia p95 para eventos de alta urgencia.
- **Inconveniente Severo de la Prioridad Estática Pura:** Si se utilizase prioridad estática sin envejecimiento, el sistema caería en el fenómeno de inanición (starvation). En un escenario de alta demanda donde los servicios prioritarios envían peticiones continuas, las tareas de prioridad baja nunca obtendrían el turno del brazo (tiempo de espera máximo $t_{e,m\acute{a}x} \rightarrow \infty$). Esto colapsaría la logística secundaria de la ciudad y reduciría el Índice de Equidad de Jain a niveles críticamente bajos.

- **La Solución Matemática: Envejecimiento Dinámico (Aging):** Nuestra implementación resuelve ambos problemas asignando una prioridad efectiva dinámica a cada pedido en cola:

$$p_{\text{efectiva}}(t) = p_{\text{base}} + (t_{\text{actual}} - t_{\text{llegada}}) / \tau$$

Donde $\tau = 8.0$ segundos representa la constante de envejecimiento calibrada. Este modelo ofrece dos garantías operativas fundamentales:

1. **Reactividad Inmediata:** Solicitudes urgentes con p_{base} alto son atendidas con latencia mínima ($p95 < 1.0$ s).
2. **Servicio Garantizado (Anti-Inanición):** Toda solicitud en espera incrementa progresivamente su prioridad efectiva a razón de $1/\tau$ puntos por segundo. Eventualmente, cualquier tarea de baja prioridad superará a peticiones recién llegadas de alta prioridad base, acotando el tiempo máximo de espera y preservando un elevado Índice de Equidad de Jain ($J \approx 0.90$).

Conclusión: La política de Prioridad Dinámica con Envejecimiento es la única que armoniza la **reactividad ante emergencias** con la **equidad operativa global**, asegurando que el recurso exclusivo de CapyTown funcione a máxima capacidad sin bloqueos ni inanición, y garantizando siempre la regla de oro de exclusión mutua a través del nodo arm_broker.